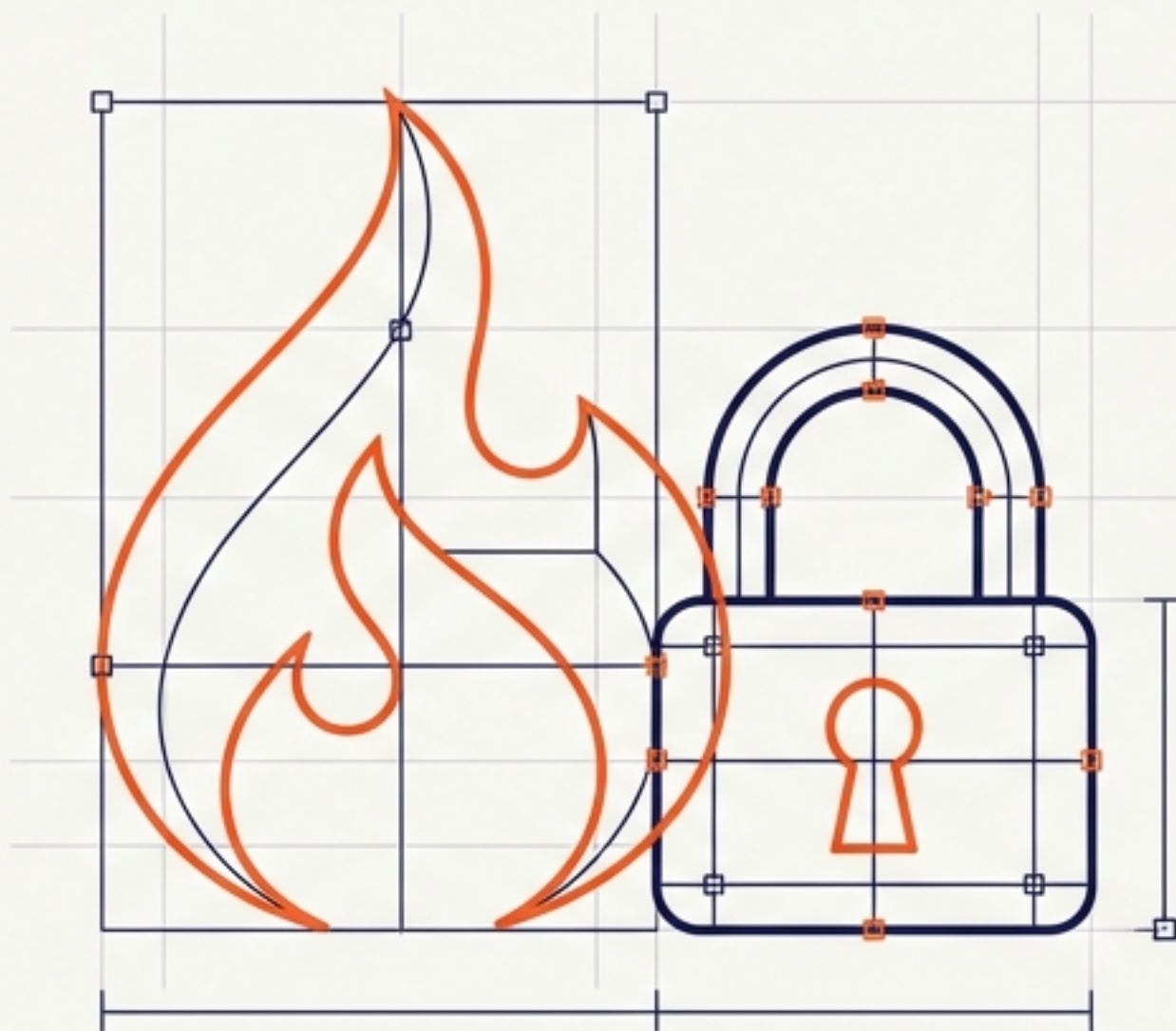




Cetak Biru Arsitektur Digital: Membangun Modul Login & Filter Keamanan

Panduan Praktis Implementasi Autentikasi Menggunakan CodeIgniter 4

Modul Praktikum Pemrograman Web 2 | Framework Lanjutan

Tujuan Pembelajaran

STATUS: INIT 



Pahami Konsep Inti

Menguasai alur logika Authentication (Autentikasi) dan Filters (Penyaring) dalam arsitektur web modern.



Rancang Sistem Login

Memahami bagaimana Sesi (Session) dan Database berinteraksi untuk memverifikasi identitas pengguna.



Implementasi CI4

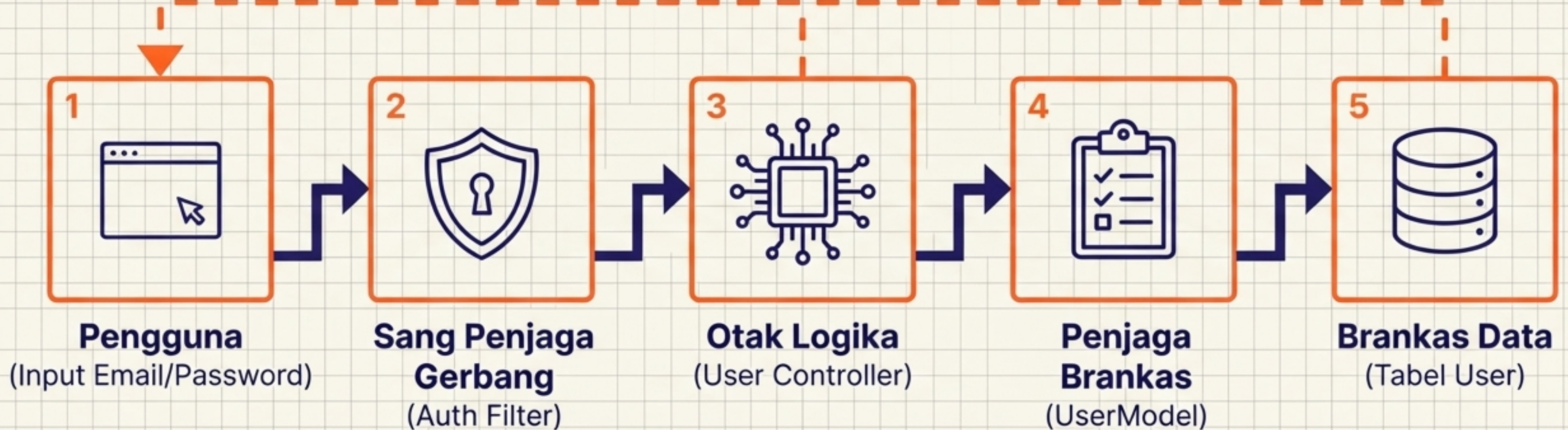
Membangun modul login fungsional dan aman menggunakan framework CodeIgniter 4 dari awal hingga akhir.



Arsitektur Keamanan

Bagaimana setiap komponen MVC dan Filter bekerja sama untuk melindungi halaman Admin.

Sesi Valid / Token



Matriks Komponen: Peran & Tanggung Jawab

Komponen	Analogi Peran	Input Utama	Output / Aksi
Login View (login.php)	Pintu Depan	Interaksi User (Ketik Email & Sandi)	Mengirim Data POST ke Controller
User Controller (User.php)	Otak Logika	Data POST dari View	Sesi Login (Sukses) ATAU Pesan Flashdata (Gagal)
User Model (UserModel.php)	Penjaga Brankas	Permintaan Query dari Controller	Mengembalikan baris data User dari Database
Auth Filter (Auth.php)	Satpam Penjaga	Permintaan Akses ke URL / admin	Mengizinkan Lewat ATAU Menendang kembali ke Login

Fondasi Data: Skema Tabel User

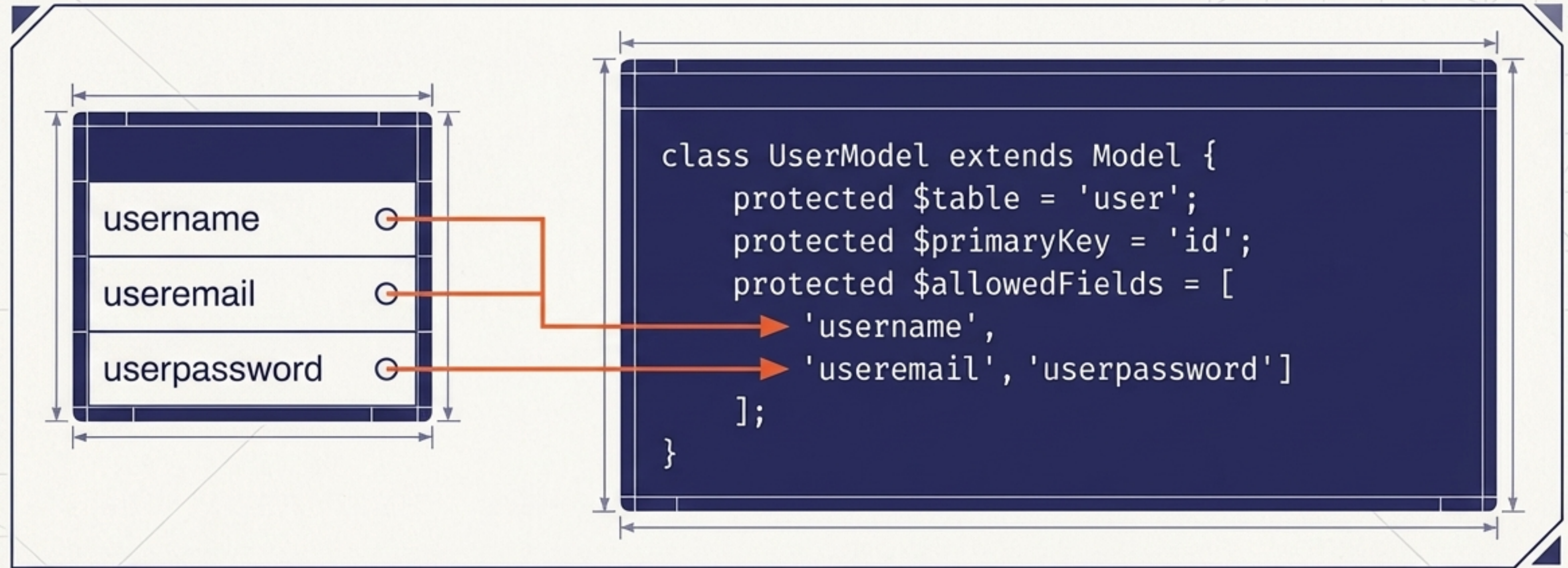
Tabel: User

id	INT(11)	PRIMARY KEY (Kunci Emas)
username	VARCHAR(200)	Label: Nama Pengguna
useremail	VARCHAR(200)	Label: Alamat Email
userpassword	VARCHAR(200)	Label: Sandi (Akan di-Hash)

```
CREATE TABLE user (  
  id INT(11) auto_increment,  
  username VARCHAR(200) NOT NULL,  
  useremail VARCHAR(200),  
  userpassword VARCHAR(200),  
  PRIMARY KEY(id)  
);
```

Fondasi Data: Sebelum logika dibangun, kita memerlukan tempat penyimpanan kredensial yang terstruktur. Tabel "user" ini adalah sumber kebenaran (source of truth) untuk proses login kita.

UserModel: Sang Penjaga Data



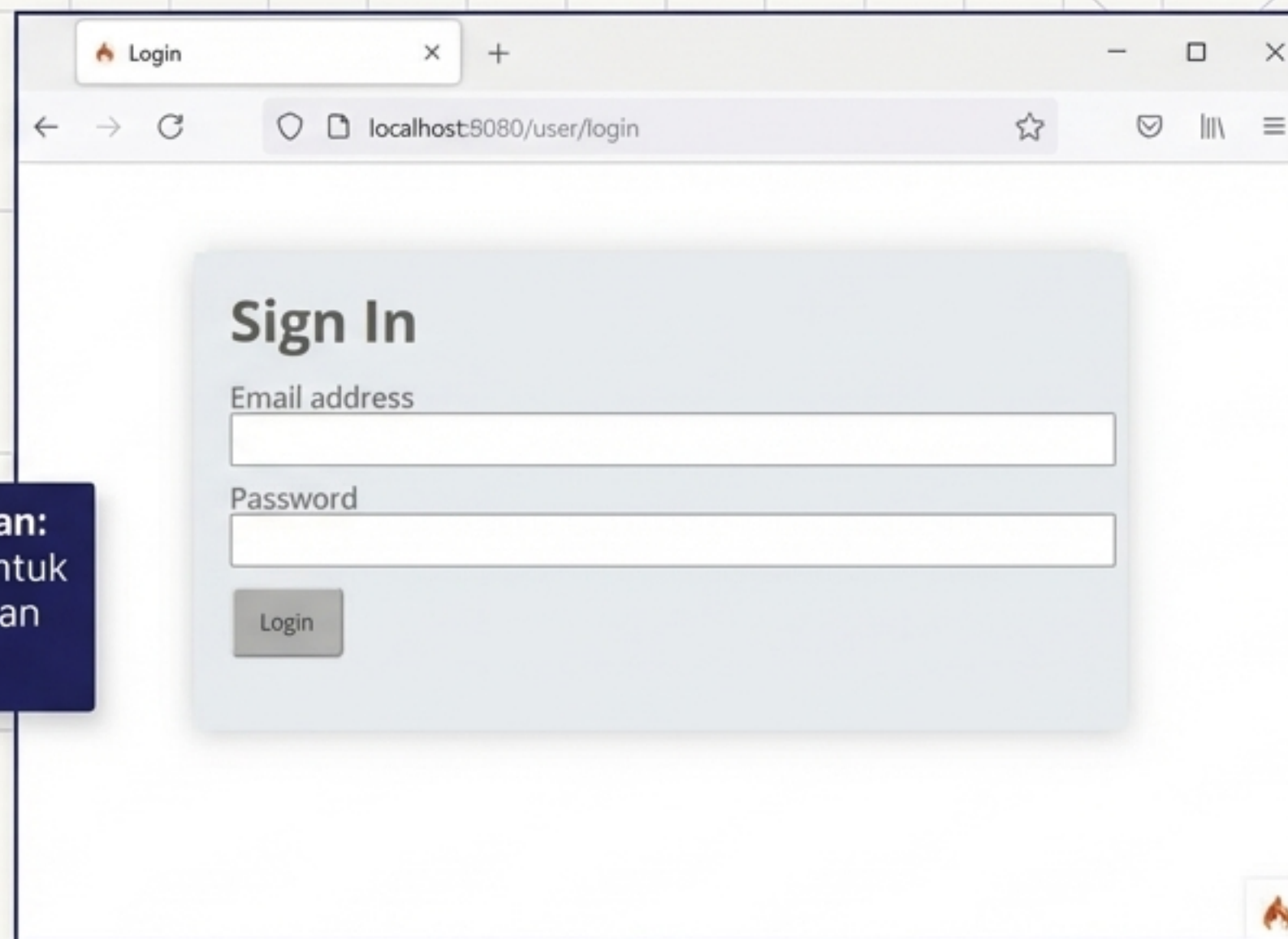
Model menjembatani aplikasi dengan database. Array **\$allowedFields** bertindak sebagai daftar tamu yang ketat—hanya kolom-kolom ini yang diizinkan untuk dibaca dan ditulis oleh sistem, mencegah manipulasi data dari luar.

Antarmuka Login (The Front Door)

```
<?php if(session()->getFlashdata('flash_msg')):?>  
    <div class="alert alert-danger">  
        <?= session()->getFlashdata('flash_msg') ?>  
    </div>  
<?php endif;?>
```

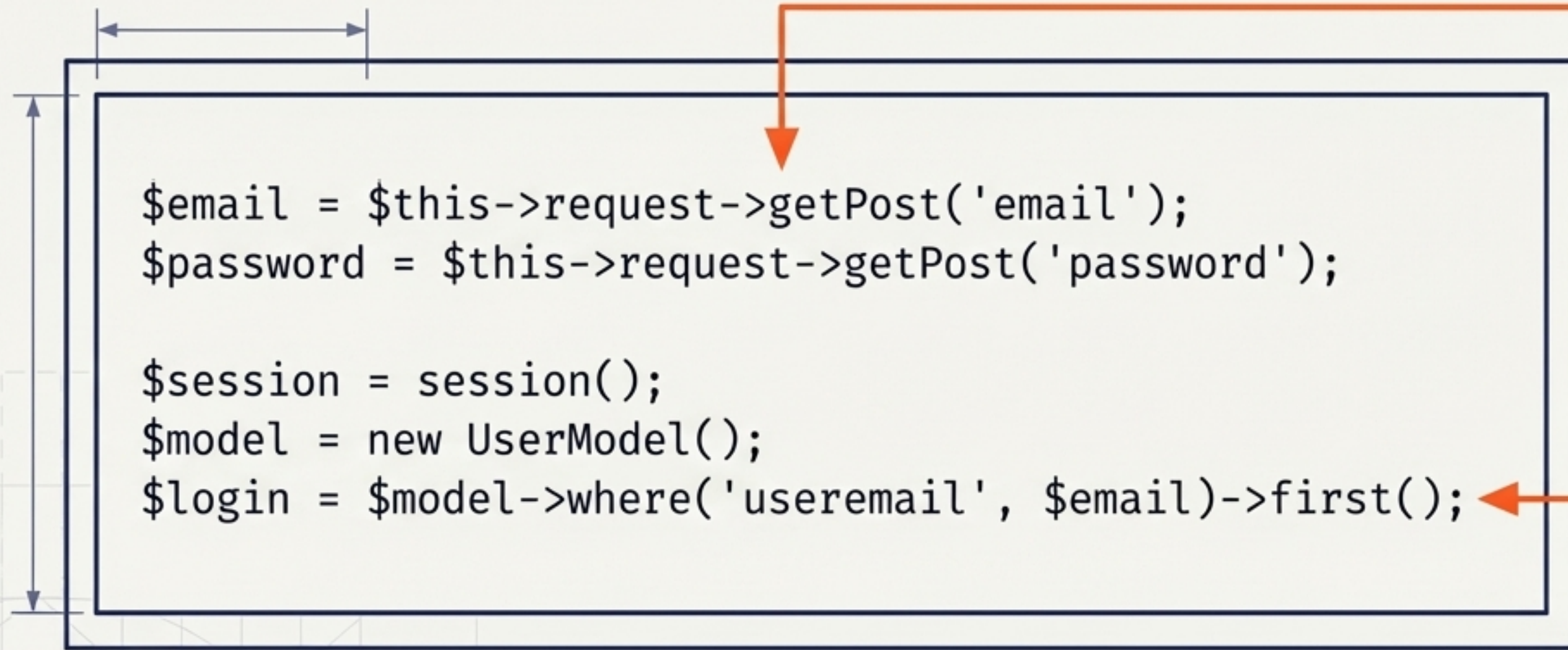
```
<form action="" method="post">  
    <!-- input email -->  
    <!-- input password -->  
    <button type="submit">Login</button>  
</form>
```

Menangkap Celah Kesalahan:
Area ini disiapkan khusus untuk menangkap dan menampilkan pesan error dari Controller.



The screenshot shows a web browser window with the title 'Login'. The address bar displays 'localhost:8080/user/login'. The main content area features a 'Sign In' form with two input fields: 'Email address' and 'Password'. Below the fields is a 'Login' button. The browser's developer tools are open, showing the HTML structure of the page. An orange arrow points from the PHP code block on the left to the 'flash_msg' variable in the HTML output, indicating how the error message is passed from the controller to the view.

Controller User: Menangkap & Mencari



The diagram shows a rectangular box representing a Controller User class. Inside the box is PHP code. An orange arrow points from the text '1. Tangkap Input' to the first two lines of code. Another orange arrow points from the text '2. Cari di Database' to the last line of code. There are also blue dimension lines on the left side of the box.

```
$email = $this->request->getPost('email');  
$password = $this->request->getPost('password');  
  
$session = session();  
$model = new UserModel();  
$login = $model->where('useremail', $email)->first();
```

1. Tangkap Input:

Controller mengambil alih data yang dikirim dari View.

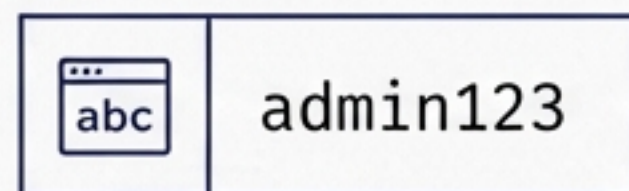
2. Cari di Database:

Memerintahkan Model untuk mencari baris data yang cocok dengan email.

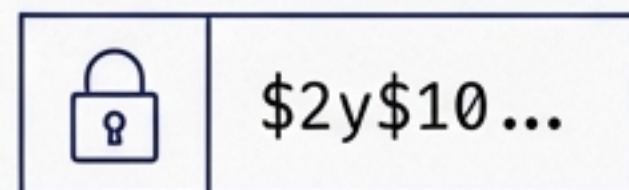
Langkah pertama otak logika adalah menangkap email dan password dari View, lalu memeriksa apakah email tersebut memiliki entri di dalam database. Jika tidak ada, akses langsung ditolak.

Anatomi Verifikasi Sandi

Input User



Hash dari Database



`password_verify($password, $pass)`

✓ TRUE

✗ FALSE

Kita TIDAK PERNAH membandingkan teks secara langsung demi keamanan. Fungsi bawaan PHP secara otomatis mencocokkan sandi mentah yang diketik pengguna dengan hash kriptografi yang tersimpan di database.

Manajemen Sesi: Output Controller

Jalur Sukses

```
$login_data = [  
    'user_id' => $login['id'],  
    'logged_in' => TRUE,  
];  
$session->set($login_data);  
return redirect('admin/artikel');
```



Sesi (Paspor Digital) Dibuat

Jalur Gagal

```
$session->setFlashdata("flash_msg", "Password salah.");  
return redirect()->to('/user/login');
```



Flashdata (Pesan Sekali Pakai)

Jika sukses, Controller mencetak 'Paspor Digital' (Session) yang berlaku selama user menjelajah. Jika gagal, Controller membuat 'Pesan Sekali Pakai' (*Flashdata*) dan menendang user kembali ke halaman login.

Sang Pembuat Kunci: Database Seeder

```
$model->insert([  
    'username' => 'admin',  
    'useremail' => 'admin@email.com',  
    'userpassword' => password_hash('admin123', PASSWORD_DEFAULT),  
]);
```

Bagaimana kita menguji login jika database kosong? Seeder menanam data dummy secara otomatis. Perhatikan bahwa kita mengamankan sandi menjadi acakan kriptografi SEBELUM menyimpannya.

```
> php spark db:seed UserSeeder
```


Skenario Uji Coba: Alur Keputusan Controller

Skenario 1 - Salah Email



Skenario 2 - Salah Sandi

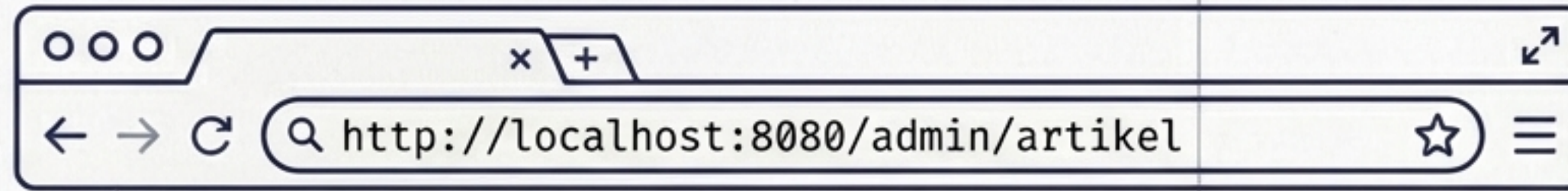


Skenario 3 - Sukses Akses



Tiga jalur utama penanganan logika otentikasi. Sistem siap menangani setiap kemungkinan input pengguna.

Celah Keamanan: Ancaman URL Bypass



Halaman Login



User



Bypass



Halaman Admin

Masalah **Celah Keamanan**:
Controller kita sudah sempurna
memverifikasi sandi.

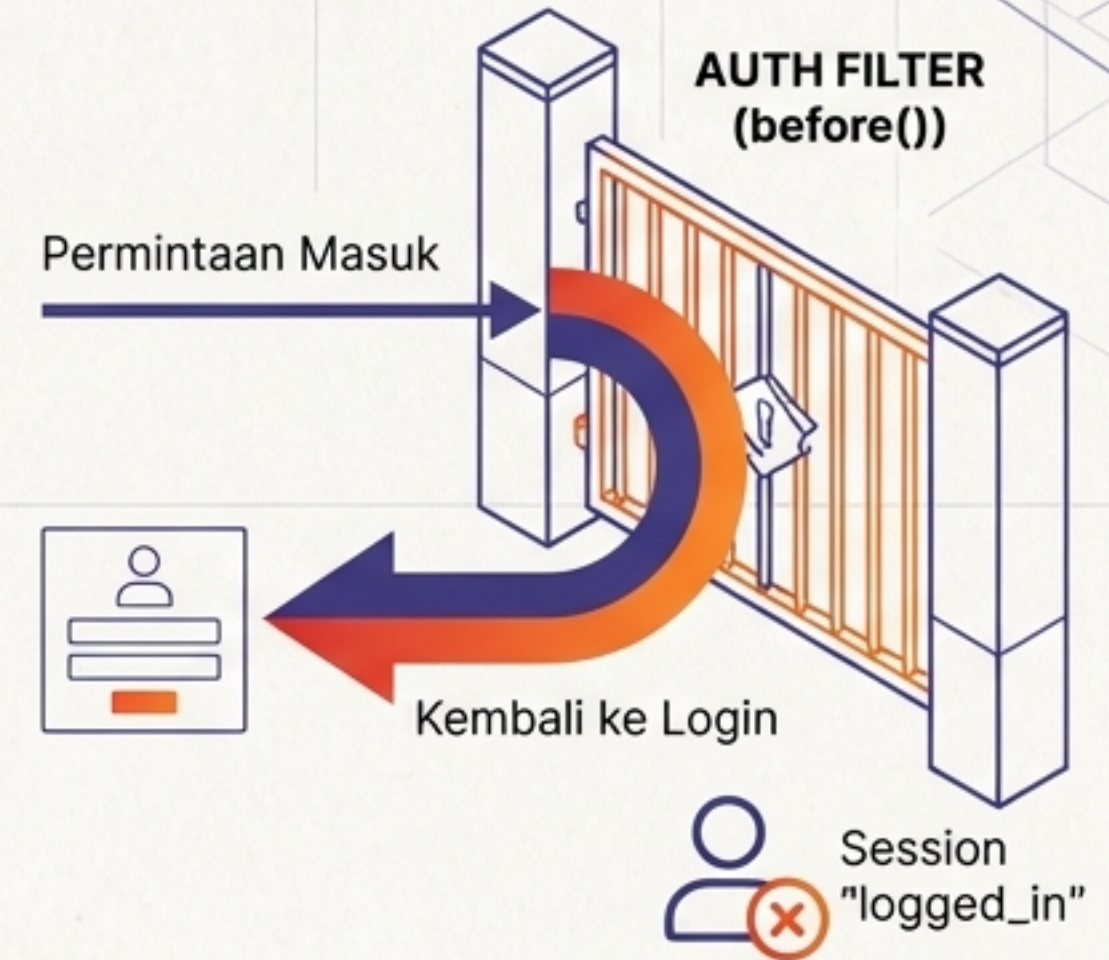
TETAPI, bagaimana jika
pengguna tidak mengklik
tombol login, melainkan
mengetikkan langsung URL
halaman admin di browser?

Tanpa proteksi tambahan,
mereka akan masuk begitu saja.

Kita butuh **Penjaga Gerbang**.

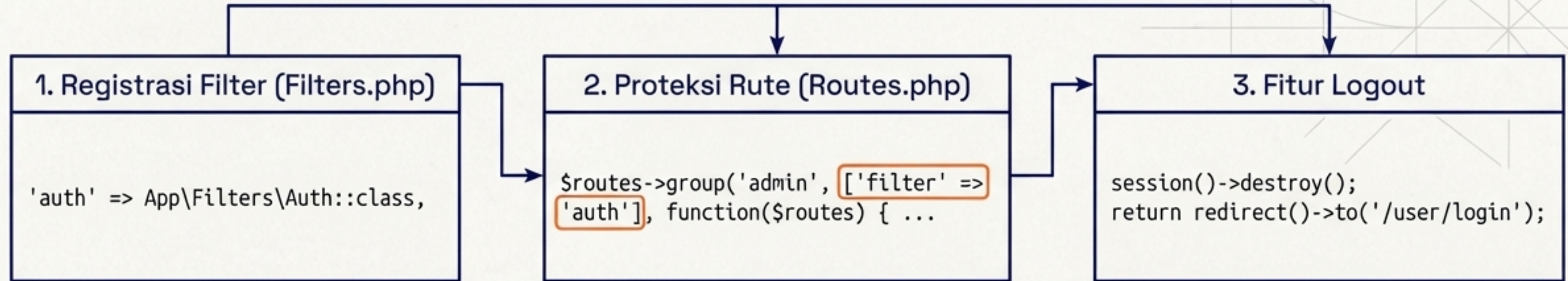
Sang Penjaga Gerbang: Auth Filter

```
public function before(RequestInterface $request, $arguments = null)
{
    // jika user belum login
    if(! session()->get('logged_in')){
        // maka redirect ke halaman login
        return redirect()->to('/user/login');
    }
}
```



Filter berjalan sebelum Controller dieksekusi. Metode **before()** mencegah setiap permintaan dan memeriksa apakah kantong pengguna memiliki tiket sesi 'logged_in'. Jika tidak ada, akses diputar balik.

Penyatuan Sistem & Misi Selesai



Misi Praktikum

- [] Lanjutkan modul pada repository Git Lab7Web.
- [] Screenshot perubahan form login & pembatasan akses.
- [] Update README.md dengan penjelasan teknis Anda sendiri.
- [] Commit dan kirim URL repository ke e-learning.